Universidad Autónoma de Ciudad Juárez

Campus CU

Subject: MFDS 2025

Team 4: Reborn

Professor: Benito Alán Ponce Rodriguez

Student Names:

Raúl Esteban Aniles Macias 222802

Villedo Martinez Sandoval 222534

Brandon Chao Diaz 219349

Ian Luis Domínguez Ramírez 222611

Erick Rangel Rubio 234591

Deadline: 18 / 11 / 2025

SRS Document

1. Introduction

Reborn is an e-commerce platform focused on the buying and selling of local crafts (Cd. Juarez). This document will cover the technical part of the project, UML diagrams, ER model, use case diagrams, etc. You'll know an in-depth understanding of how the project is intended to be carried out in detail, where everything the project needs to become a quality product will be described.

The aim of this product is to create a platform where artists can share and sell their work, as well as buyers interested in unique catalog pieces.

1.1 Purpose

The purpose of this SRS document is to give a comprehensive description of the software product. Our goal in this document is therefore to provide to the stakeholders a detailed description about the product's features; what the system will do, the product's infrastructure, the requirements and constraints within the system.

1.2 Scope

The Reborn platform is a web-based e-commerce platform conceived as a circular economy marketplace. Its main purpose is to connect local artists and designers (from Ciudad Juárez) who specialize in creating design pieces made from recycled or second-hand materials.

The main functionalities planned for the project are:

- **Profile management**: User accounts that allow buying and selling.
- **Product catalog**: A marketplace where sellers can publish, manage, and sell their unique pieces.

The application will not include international shipping logistics.

1.3 Definitions, acronyms, and abbreviations

Term	Definition
SRS	Software Requirements Specification
e-commerce	Process of buying and selling goods and services over the internet
UML	Unified Modeling Language
ER	Entity Relationship Model
AWS	Amazon Web Services
STRIPE	It is a set of APIs that helps online payment processing and e-commerce solutions for internet businesses of all sizes.

1.4 References

[1]. IEEE. IEEE Std. 830-1998 IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf

1.5 Overview

This Software Requirement Specification document presents a structured and comprehensive description of the proposed system. It is divided into four main sections. The first section provides an introduction of the project, outlining its purpose, scope, and key concepts. The second section offers a description of the system including its context, main functions, user characteristics, constraints, assumptions, and business rules. The third section specifies the system's functional and non-functional requirements, external interfaces, database design, and software attributes. Finally, section four includes supporting materials such as appendices, glossary terms, and references that facilitate understanding and future development of the system.

2. Overall description

2.1 Product perspective

Reborn is a web and mobile platform designed to support the creativity of local artisans and designers, transforming waste materials or second-hand products into unique design pieces. The platform aims to provide a reliable space where users can order, buy, and discover unique catalog objects with both aesthetic and functional value.

To make it easier to choose the right artist, Reborn shows details about each creator, including their experience, specialties, previous projects, and reviews from other customers, helping users make informed decisions.

Reborn also offers a catalog section where users can explore pieces already available for purchase, with filters by type of object, material, and style, and an interactive shopping cart to select products. The platform also ensures secure transactions through Stripe.

2.2 Products functions

2.2.1 General use case diagram

This use case includes the actions related to accessing the platform.

Actor	**Description**
Registered User	Type of user who has completed the registration process in the system. This role has access to all core system functionality and advances features that require customization or history.
Unregistered User	This user interacts with the system without being logged in.
Database	System component that stores, organizes, manages all system information. It provides the required data for system operations.

Product Management and Catalog:
This use case covers all actions related to products and browsing. Sellers or artists can create and edit their unique products so they can be shown on the platform.

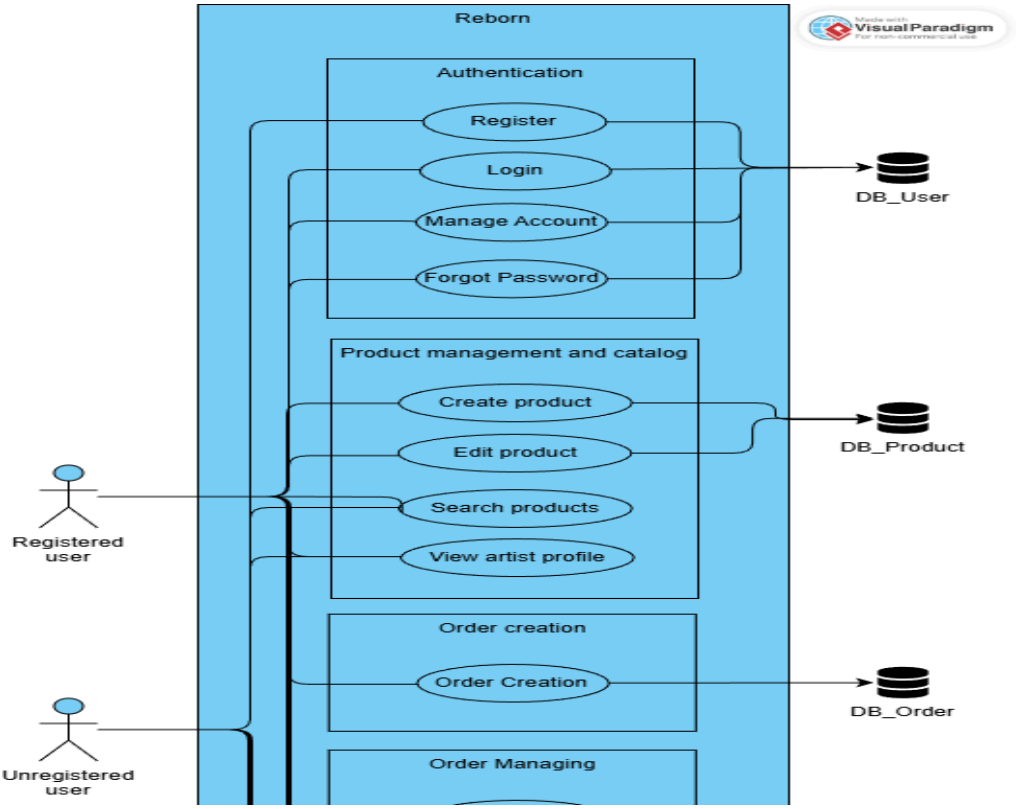
Users, both registered and non-registered, can search for products using filters or categories, and they can also view the public profile and portfolio of any artist.

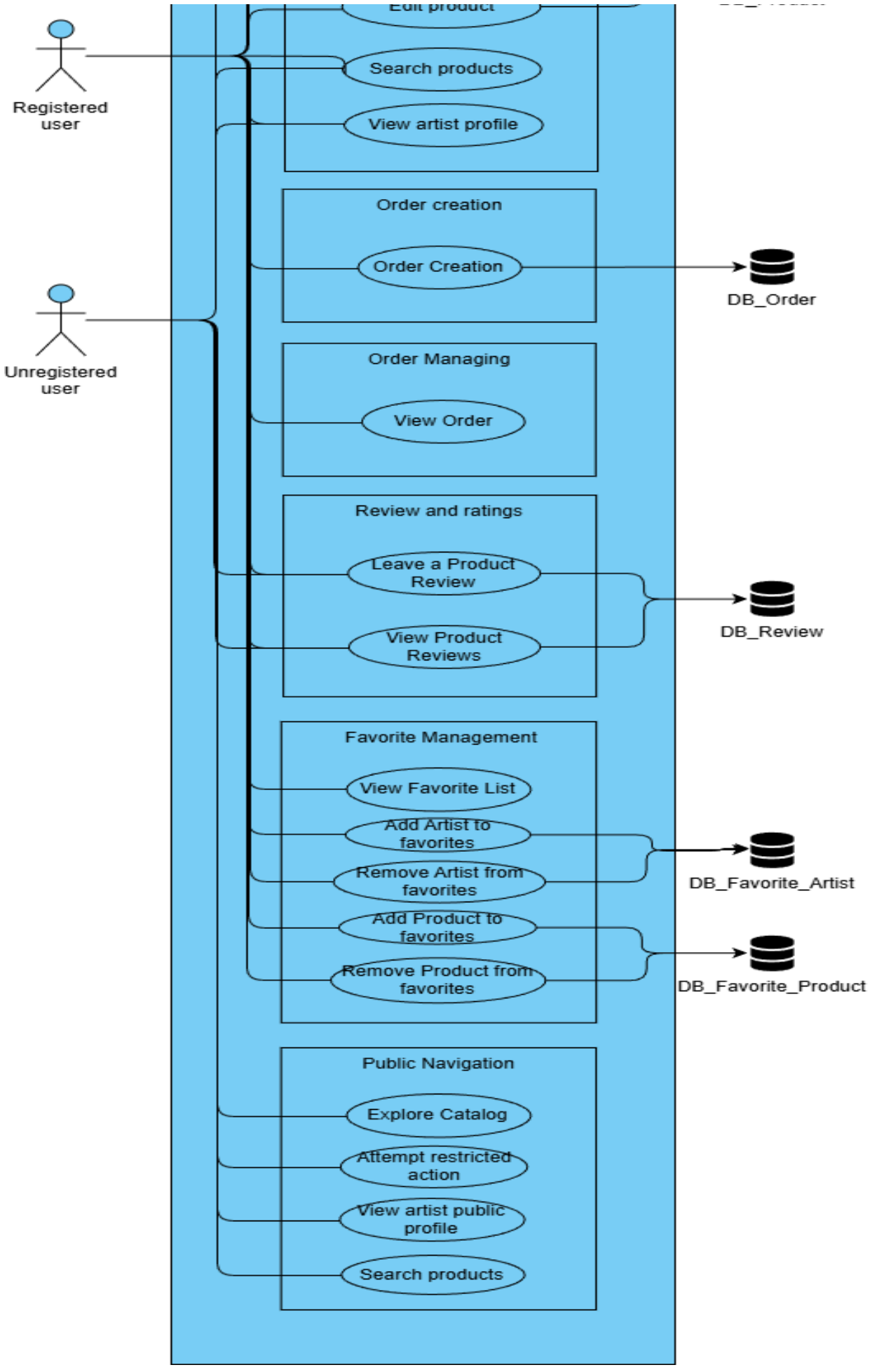
Order Creation and Managing:
This use case handles the process of buying a product. Registered Users can begin an order by selecting the items they want to purchase, and they can later check the status of their order to follow its progress from start to finish.

Reviews and Ratings:
This use case allows Registered Users who have completed a purchase to leave a comment and a rating about the product they bought. All users, including non-registered ones, can view the reviews and ratings to know other customers' opinions.

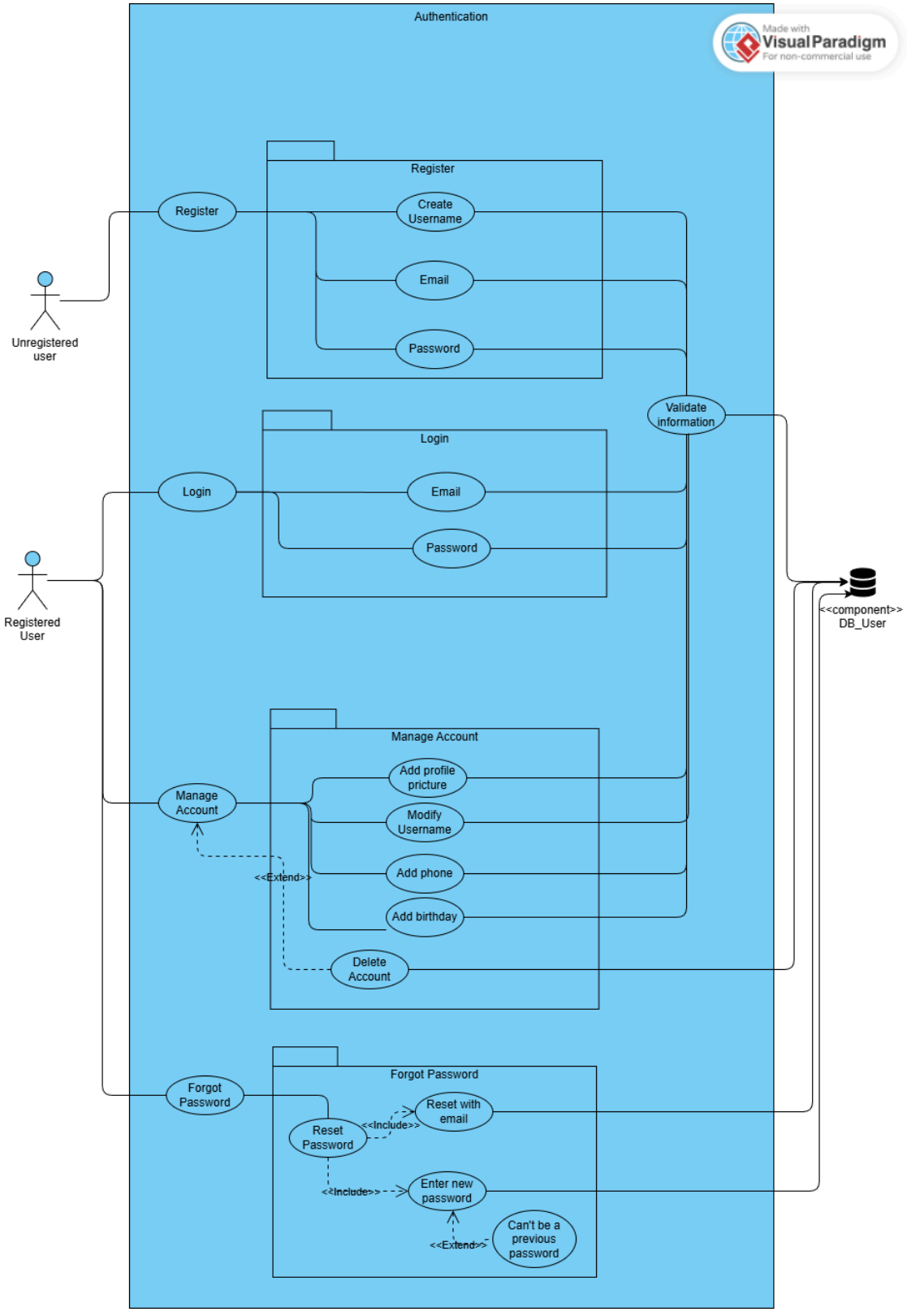
Favorite Management:
This use case allows Registered Users to save artists or products they like into a personal favorite list. They can later view this list or remove any artist or product they no longer want to keep saved.

Public Navigation:
This use case includes the actions that Non-Registered Users can do without creating an account. They can explore the catalog, see products, and view artist public profiles. If they try to access a restricted action, such as starting a purchase or adding a favorite, the system will ask them to log in or register.




Edit product
Search products
View artist profile
Registered user
Order creation
Order Creation
DB_Order
Unregistered user
Order Managing
View Order
Review and ratings
Leave a Product Review
View Product Reviews
DB_Review
Favorite Management
View Favorite List
Add Artist to favorites
Remove Artist from favorites
DB_Favorite_Artist
Add Product to favorites
Remove Product from favorites
DB_Favorite_Product
Public Navigation
Explore Catalog
Attempt restricted action
View artist public profile
Search products

2.2.2 Module 1: Authentication



Use case name:	CU-101: Register

Description:	Unregistered users can register a new account.
Actors:	Unregistered user, Database
Preconditions:	The user doesn't have a registered account.
Main flow:	1. The user selects the "Register" option. 2. The user enters the required information for each field to be stored into their new account (unique username, email, and password). 3. The user submits the information required by the system. 4. The system validates if each field was filled correctly. 5. The account gets created with the user information.
Alternative flow:	● A1: Invalid information. ○ If one of the fields gets filled with invalid input from the user, the system shows in the field a message that indicates that it has invalid information.
Postconditions:	If every field gets filled with the appropriate information, the system will display a success message for a newly registered account and take the user to the main page. Or else, the system will perform what it was explained in **A1**.

Use case name:	**CU-102: Log in**
Description:	Registered users can log into their account with their email and password provided in **UC-101**.
Actors:	Registered User, Database
Preconditions:	The user must already have a registered account.
Main flow:	1. The user selects the "Log in" option. 2. The user enters both the email and password assigned to the previously registered account.. 3. The system validates if each field was filled correctly.
Alternative flow:	● A1: Invalid information. ○ If one of the fields gets filled with invalid input from the user, the system shows in the field a message that indicates that it has invalid information. ● A2: Information doesn't belong to a registered account. ○ If one or both fields get filled with information that doesn't belong to a registered account, the system will show an error message. ○ If the user exceeds the allowed number of consecutive failed login attempts, the system will block the account temporarily.

Postconditions:	If a user inputs the information belonging to a registered account, the system will display a success message and take the user to the main page. Or else, the system will perform A1 or A2 based on the user input.

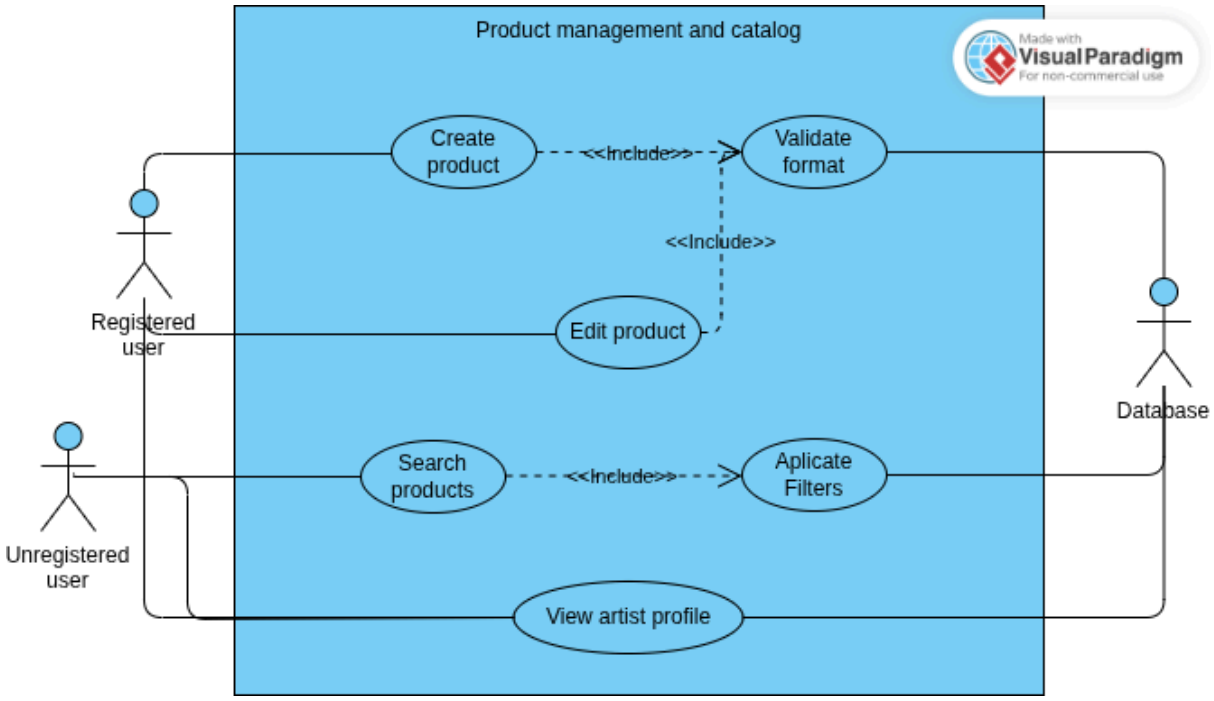
Use case name:	**CU-103: Password Recovery**
Description:	In case a user forgets their password, selecting the “Forgot password?” option will allow users to restore it to be able to access their account.
Actors:	Registered User, Database, Email Service.
Preconditions:	The user must already have a registered account.
Main flow:	1. The user selects the “Forgot password?” option from the login page. 2. The system asks the user to input their email address. 3. The system validates that the email exists in the database. 4. The system sends an email to proceed with the password restoration process.
Alternative flow:	● A1: Inputs in an invalid format. ○ The system won’t be able to verify if the email address is in the database because the input isn’t in the specified format. ● A2: User’s input doesn’t belong to a registered account ○ The email address doesn’t belong to an already registered account in the database. The system will throw an error message telling the user that the information isn’t linked to any account.
Postconditions:	When introducing correctly either the email or phone number, belonging to a registered account, the system will show a message to the user telling them that either an email or text message has been sent successfully, so the user can proceed with the password restoration process.

Use case name:	**CU-104: Manage Account**
Description:	Registered users can manage their account information to properly check what’s stored under each field and add additional details such as phone number, biography, or profile picture.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in.

Main flow:	1. The user selects the “Profile” option. 2. The user can check every field of their information. 3. The user may select a field to update its information.
Alternative flow:	None.
Postconditions:	If changes are made to a field properly, saving these changes will display a success message to the user. If no changes get made to any field, the information will stay just as it already is.

Use case name:	**CU-105: Delete Account**
Description:	Registered users have the option to delete their account in any case in the “Profile” section.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in.
Main flow:	1. The user selects the “Profile” option 2. The user selects the “Delete Account” option. 3. The system displays a message explaining that all data will be permanently removed and asking “Are you sure?” and giving two options “No” and “Yes”. 4. The user confirms the deletion. 5. The system deletes all user-related data in the database. The system displays a success message confirming the account has been deleted. 6. The user is redirected to the home page.
Alternative flow:	● A1: User cancels the deletion: ○ The user selects “No” in the confirmation dialog. The system aborts the process and redirects the user to the “Profile” page.
Postconditions:	The user’s account and related data are permanently removed. The user cannot login again.

2.2.3 Module 2: Product management and catalog



Use case name:	**CU-201: Create product**
Description:	As an artist/seller, I want to create products in my catalog
Actors:	Registered User, Database.
Preconditions:	The user must already have a registered account and logged in.
Main flow:	1. Artist navigates to "My Products" section in dashboard 2. System displays product creation form 3. Artist enters product details: name, description, price, category, stock quantity 4. Artist uploads product images (multiple allowed) 5. System validates image formats and file sizes 6. System saves product data to PostgreSQL database 7. System uploads images to Amazon S3 bucket 8. System returns success confirmation 9. Product becomes visible in public catalog
Alternative flow:	● 5a. Invalid image format: System rejects upload and displays supported formats ● 5b. File size exceeds limit: System automatically optimizes image ● 5c. Required fields missing: System highlights missing fields and prevents submission

	● 7a. S3 upload fails: System retries 3 times, then shows error message
Postconditions:	1. Product is created and published in catalog 2. Product appears in artist's management panel 3. Product is searchable and visible to all users

Use case name:	**CU-202: Validate image format**
Description:	System validates uploaded images for supported formats and specifications
Actors:	Registered User, Database.
Preconditions:	1. Artist has initiated product creation or editing 2. Image files have been selected for upload
Main flow:	1. System receives image files from frontend 2. System checks file extensions (.jpg, .jpeg, .png, .webp) 3. System verifies MIME types match extensions 4. System checks file sizes (max 10MB per image) 5. System validates image dimensions (min 500x500px, max 3000x3000px) 6. System returns validation result 7. If valid, proceeds to upload process
Alternative flow:	● 3a. MIME type mismatch: System rejects file and notifies user ● 4a. File too large: System either rejects or triggers optimization process ● 5a. Dimensions too small: System warns user but allows proceeding ● 5b. Dimensions too large: System automatically resizes image
Postconditions:	1. Images are either accepted or rejected with specific reasons 2. Valid images proceed to storage process 3. Invalid images are removed from upload queue

Use case name:	**CU-203: Edit product**
Description:	Allows artists to modify existing product information and images.
Actors:	Registered User, Database.
Preconditions:	1. Artist must be logged in 2. Product must exist and belong to the artist 3. Product must not have pending orders
Main flow:	1. Artist accesses "My Products" management panel 2. System displays list of artist's products 3. Artist selects product to edit

	4. System loads current product data into edit form 5. Artist modifies desired fields (name, description, price, etc.) 6. Artist can add/remove product images 7. System validates changes and new images 8. System updates product in database 9. System manages S3 storage (adds/removes images as needed) 10. System confirms successful update
Alternative flow:	● 3a. Product not found: System shows error and returns to product list ● 6a. Remove main image: System requires selecting new main image first ● 7a. Price reduction: System notifies customers with pending orders ● 9a. Storage update fails: System rolls back database changes
Postconditions:	1. Product information is updated across the platform 2. Search indexes are updated if relevant fields changed 3. Customers see updated product information immediately

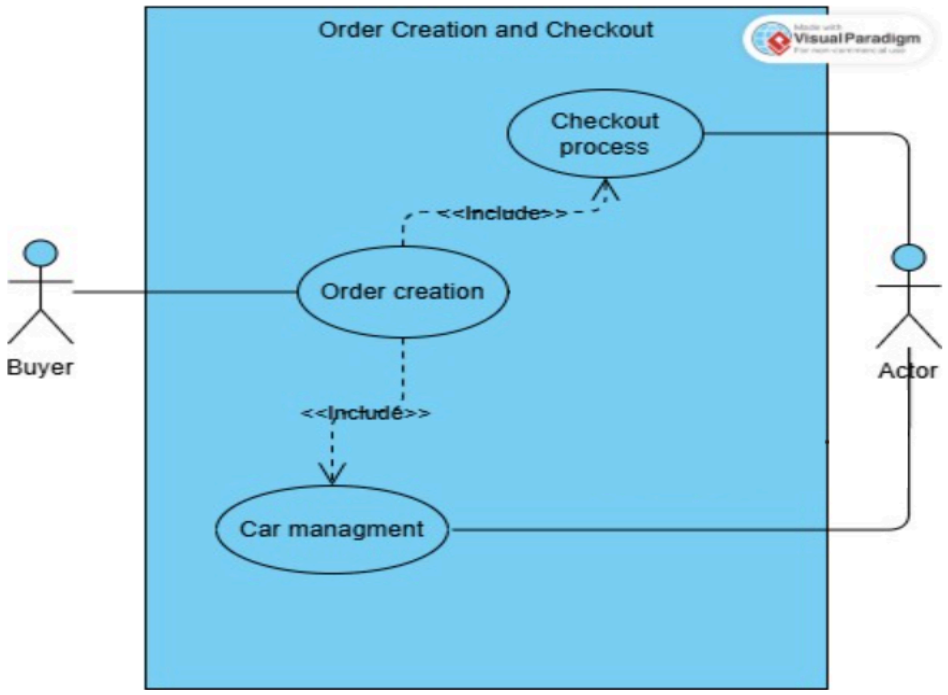
Use case name:	**CU-204: Search products**
Description:	Enables users to find products using text search and various filters.
Actors:	Registered and Registered User, Database.
Preconditions:	1. User has access to the platform (registered or guest) 2. Catalog has products available
Main flow:	1. User enters search terms in search bar 2. System queries database for matching products 3. System applies relevance scoring to results 4. System returns paginated results (20 products per page) 5. User views search results with product cards 6. User can apply additional filters 7. System updates results based on applied filters 8. User navigates through result pages
Alternative flow:	● 2a. No search terms: System shows featured or recent products ● 2b. Complex query: System uses full-text search with stemming ● 4a. Many results: System shows most relevant first ● 7a. No results: System suggests similar terms or categories
Postconditions:	1. Search results are displayed to user 2. Search query is logged for analytics 3. CloudFront may cache popular search results 4. User can refine search or view product details

Use case name:	**CU-205: Apply filters**
Description:	Allows users to refine product search results using multiple criteria
Actors:	Registered and Registered User, Database.
Preconditions:	1. User has performed a search or is browsing categories 2. Search results are currently displayed
Main flow:	1. User opens filter panel 2. System displays available filter options: a. Price range b. Categories (checkboxes) c. Artists (checkboxes) d. Rating (star selection) e. Available (in stock) 3. User selects desired filter criteria 4. System applies filters to current result set 5. System updates product count and display 6. User views filtered results
Alternative flow:	● 4a. No products match filters: System shows empty state with filter reset option. ● 4b. Complex filter COmbination: System optimizes database query ● 5a. Many filters active: SYstem shows active filter tags with remove options ● 5b. Performance issues: System uses cached filtered results when available.
Postconditions:	1. Filtered results are displayed 2. Active filters are visible and manageable 3. URL updates with filter parameters for sharing 4. User can continue refining or clear filters

Use case name:	**CU-206: View artist profile**
Description:	Displays public artist profile including biography, portfolio and contact information.
Actors:	Registered and Registered User, Database.
Preconditions:	1. Artist profile exists and is public 2. User has navigate to artist profile page
Main flow:	1. User accesses artist profile via search, product page, or direct link. 2. System loads information: a. Biography and background b. Profile picture c. Contact information (if public)

	d. Social media links 3. System loads artist's product portfolio 4. System displays reviews and ratings for artist 5. User browses artist's products and information 6. User can follow artist or view products
Alternative flow:	● 2a. Artist not found: System shows 404 error page ● 2b. Profile not public: System shows limited information or access denied ● 3a. No products: System shows empty portfolio message ● 6a. User not logged in: System prompts registration for follow feature
Postconditions:	1. Artist profile is displayed with complete information 2. Artist's product portfolio is accessible 3. User can navigate to artist's products or contact artist

2.2.4 Module 3: Order Creation



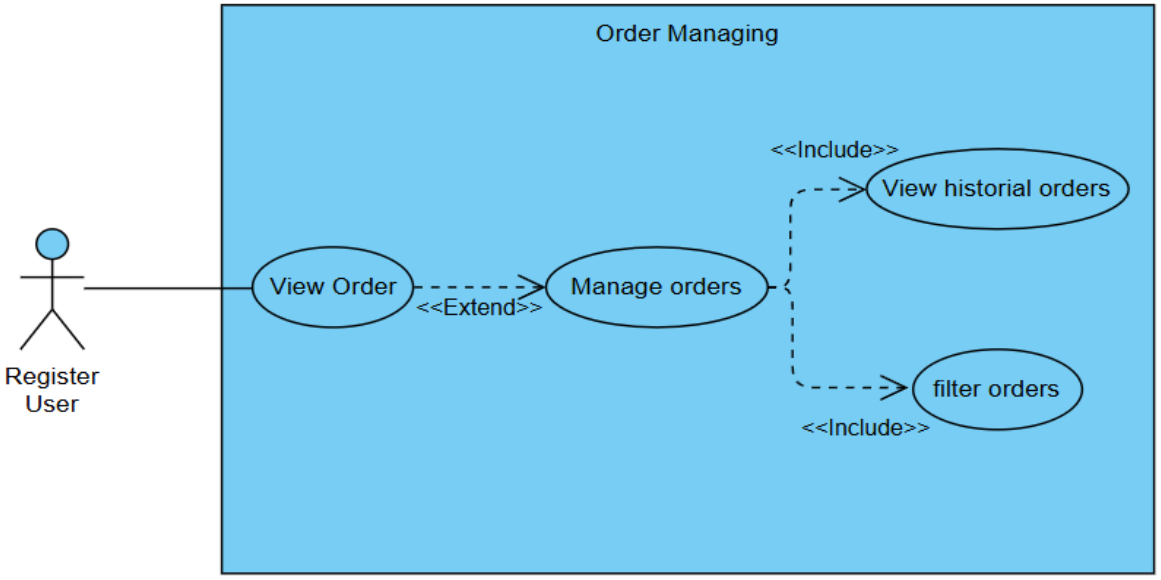
Use case name:	**CU-301: Order Creation**
Description:	As a buyer, the user can place orders either from the existing catalog.
Actors:	Buyer, database
Preconditions:	1. The buyer must be logged in. 2. There must be available products in the catalog.
Main flow:	1. The buyer navigates to the product catalog. 2. The buyer selects a product from the catalog 3. The system displays the product details. 4. The buyer provides the necessary information, including: ● Payment method 5. The system validates all fields. 6. The buyer confirms and submits the order. 7. The system processes the order, generates a unique order ID, and stores it in the database. 8. The system sends a confirmation message.
Alternative flow:	● **A1: Missing or invalid information** If required fields are incomplete or invalid, the system highlights errors and prevents submission. ● **A2: Payment error** If payment cannot be processed, the system shows an error

	message and allows the buyer to retry. ● **A3: Product unavailable** If a catalog product goes out of stock, the system displays an "Unavailable" message and removes it from the order.
Postconditions:	1. The order is successfully created and stored in the system. 2. The buyer receives confirmation and can view the order in the "My Orders" section.

Use case name:	**CU-302: Checkout Process**
Description:	As a buyer, the user can complete the purchase securely.
Actors:	Buyer, database
Preconditions:	1. The buyer has at least one product in their cart. 2. The system and payment gateway (Stripe) are operational.
Main flow:	1. The buyer opens the cart and proceeds to checkout. 2. The system displays an order summary, including product details, total price, and taxes if applicable. 3. The buyer provides or confirms shipping address and payment method. 4. The system establishes a secure connection (SSL) for payment processing. 5. The buyer confirms the payment through Stripe. 6. Stripe validates and processes the payment securely. 7. The system receives confirmation from Stripe and finalizes the order. 8. The buyer receives immediate payment and order confirmation via email and on-screen message.
Alternative flow:	● **A1: Payment declined** Stripe returns an error, and the system notifies the buyer to try again or use another payment method. ● **A2: Connection or timeout error** The system informs the user that the payment could not be processed and allows retry. ● **A3: Invalid card information** The buyer is prompted to re-enter or select a different payment option.
Postconditions:	1. The payment is processed successfully via Stripe. The order status changes to "Paid." 2. No payment information is stored on the platform.

Use case name:	**CU-303: Cart Management**
Description:	As a buyer, the user can manage the products in my shopping cart.
Actors:	Buyer, database
Preconditions:	1. The buyer must be logged in. 2. The cart must exist and contain products or be ready to receive them.
Main flow:	1. The buyer adds products to the shopping cart from the catalog. 2. The buyer can view all products currently in the cart. 3. The buyer can update quantities or remove items. 4. The system updates the cart totals automatically. 5. The cart is saved and remains persistent between user sessions.
Alternative flow:	● **A1: Product out of stock** The system removes the unavailable product from the cart and notifies the user. ● **A2: Invalid quantity** The system prevents setting quantities below 1 or above available stock.
Postconditions:	1. The cart is updated according to user actions. 2. The system maintains a persistent cart for each buyer.

2.2.5 Module 4: Order managing



Use case name:	**CU-401: View orders**
Description:	Users can view their orders in the pending orders view.
Actors:	Registered user
Preconditions:	The user must have at least one order, either completed or pending.
Main flow:	1. The user selects the pending orders view. 2. The user can review their orders.
Alternative flow:	1. A1: Does not have any orders a. The user cannot access the pending orders view if they have no orders.
Postconditions:	The system displays a list of orders owned.

Use case name:	**CU-402: Manage orders**
Description:	Users can manage their orders in the orders view
Actors:	Registered user
Preconditions:	The user must have at least one order, either completed or pending.
Main flow:	1. The user selects the orders view. 2. The user can review and manage their orders.

Alternative flow:	1. A1: Does not have any orders a. The user cannot access the orders view if they have no orders.
Postconditions:	The system displays the orders view.

Use case name:	**CU-403: Filter orders**
Description:	Users can filter the list of orders to get a more organized view.
Actors:	Registered user
Preconditions:	The user must have at least one order, either completed or pending.
Main flow:	1. The user goes to the order view 2. Select the filter icon 3. Choose whichever filter they want 4. The filter is applied to the list
Alternative flow:	A1: Does not have any orders The user cannot access the orders view if they have no orders.
Postconditions:	The filter is applied to the list

Use case name:	**CU-407: View order details**
Description:	Users can view order details from their orders list
Actors:	Registered user
Preconditions:	The user must have at least one order, either completed or pending
Main flow:	1. The user goes to the order view 2. Select whichever order they want to see on detail 3. The system display a new page containing the details of the selected order
Alternative flow:	A1: Does not have any orders The user cannot access the orders view if they have no orders.
Postconditions:	The system displays a page containing the selected order details

2.2.6 Module 5: Review and Rating System

Use case name:	**CU-501: Leave Product Review**
Description:	Allows verified purchasers to submit reviews and ratings for purchased products
Actors:	Registered user, database
Preconditions:	1. User must be registered and logged in 2. User must have completed purchase of the product 3. Order must be in “delivered” or "completed" status 4. User hasn’t already reviewed this specific purchase
Main flow:	1. User navigates to “My Orders” section 2. System displays list of completed orders 3. User selects “Write Review” for specific product 4. System opens review from with: a. Star rating (1-5 stars) b. Review title (optional) c. Detailed review text 5. User submits review with rating 6. System validates purchase eligibility 7. System saves review to database 8. System updates product’s average rating 9. System confirms review submission

Alternative flow:	● 3a. Review period expired: System informs user review window has closed ● 6a. Invalid purchase: System rejects review and explains eligibility criteria ● 7a. Inappropriate content: System flags for moderation before publishing
Postconditions:	1. Review is published on product page 2. Product's average rating is calculated 3. User's review history is uploaded 4. Review becomes visible to other users after moderation

Use case name:	**CU-502: Validate Purchase**
Description:	Verifies that a user actually purchased the product they're attempting to review
Actors:	Database, system
Preconditions:	1. User is attempting to submit a product review 2. User is authenticated 3. Product ID and User ID are available
Main flow:	1. System receives review submission request 2. System queries orders database for: a. User ID match b. Product ID match c. Order status = "completed" or "delivered" d. Purchase date within review period (90 days) 3. System checks if review already exists for this purchase 4. System returns validation result (true/false) 5. If valid, proceeds with review processing
Alternative flow:	● 2a. Multiple Purchases: System allows review for most recent purchase ● 2b. Purchase outside review period: System returns "period expired" ● 3a. Existing review: System offers to edit existing review instead ● 4a. Validation timeout: System uses cached validation results
Postconditions:	1. Purchase validation is confirmed or denied 2. Appropriate message is returned to user 3. Audit log records validation attempt 4. System proceeds with review submission or shows error

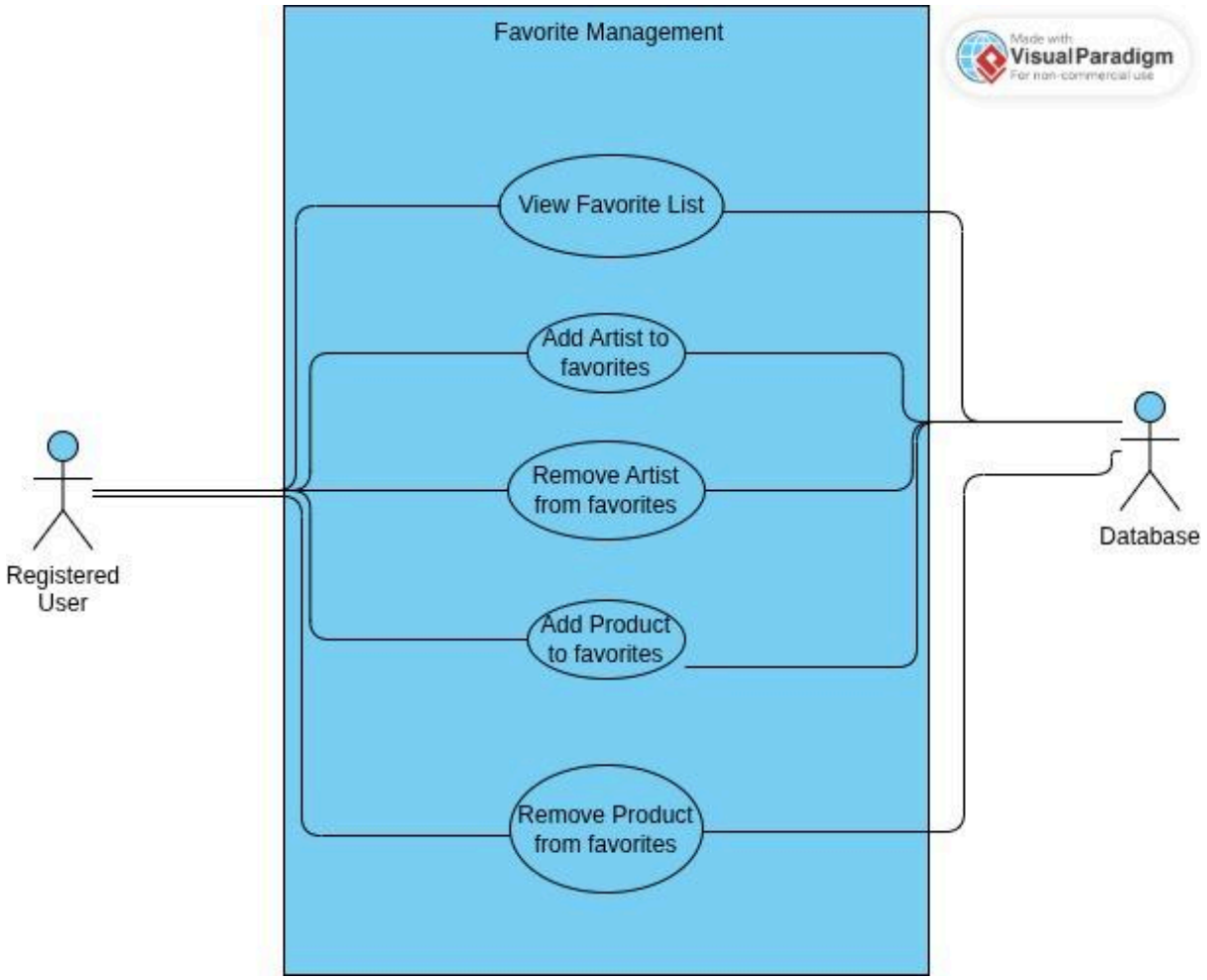
Use case name:	**CU-503: View product Review**
Description:	Displays all published reviews and ratings for a specific product

Actors:	Registered and unregistered user and database
Preconditions:	1. Product Exists and is published 2. User is viewing product details page
Main flow:	1. User navigates to product details page 2. System loads product reviews section 3. System displays: a. Average rating and distribution b. Most helpful reviews first c. Paginated review list (10 per page) d. Review statics (total count, verified purchases) 4. User reads reviews and ratings 5. User can mark reviews as helpful/unhelpful 6. User can filter and sort reviews
Alternative flow:	● 3a. No reviews: System shows “no reviews yet” message ● 3b. Many reviews: System uses infinite scroll instead of pagination ● 5a. User not logged in: System tracks helpful votes anonymously ● 6a. Complex filters: System shows loading indicator during filtering
Postconditions:	1. Product reviews are displayed to user 2. Helpful vote counts are updated 3. User engagement metrics are recorded 4. USer can proceed to purchase or browse other products

Use case name:	**CU-504: FIlter reviews**
Description:	Allows users to sort and filter product reviews based on various criteria
Actors:	User and database
Preconditions:	1. User is viewing product to reviews 2. Multiple review exist for the product
Main flow:	1. User opens review filter/sort options 2. System displays available filters: a. Rating (1-5 stars) b. Sort by: Most recent, Most helpful, HIghest rating, Lowest rating c. Verified purchases only d. Search within reviews 3. User selects filter criteria 4. System applies filters to review list 5. System updates displayed reviews 6. User views filtered reviews
Alternative flow:	● 4a. No reviews match filters: System shows empty state with filter reset option

	● 4b. Performance issues: System uses client-side filtering when possible ● 5a. Complex search terms: System uses full-text search on review content ● 5b. Many active filters: System shows filter summary with clear options
Postconditions:	1. FIltered review list is displayed 2. Active filters are visible and can be modified 3. URL parameters are uploaded for shareable filtered views 4. User can continue reading or adjust filters

2.2.7 Module 6: Favorite List Management



Use case name:	**CU-901: View Favorite List**
Description:	Allows the user to view their favorite list of products and artists.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in.
Main flow:	1. The Registered User selects the “Favorite List” option (Heart Icon). 2. The system consults the database and retrieves all the products and artists that the user has marked as favorite. 3. The system displays the list in an organized way with separate tabs for “Artists” and “Products”. 4. For every Artist, the system displays their image, username, and if clicked the system redirects the user to their page. 5. For every Product, the system displays its image, name, price and if clicked, the system redirects the user to the page of the product.
Alternative flow:	● A1: Favorite list is empty.

	○ The system displays a message indicating that the user has not saved any artist or products.
Postconditions:	The Registered User has seen their list of favorite products and artists.

Use case name:	**CU-902: Add artist to favorites**
Description:	Allows the user to add an artist to their favorites list.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in.
Main flow:	1. The Registered User browse through the artists section 2. The Registered User clicks on the “Add to favorites” option (Heart Icon). 3. The system stores the relationship between the Registered User ID and the artist ID in the database. 4. The system updates the favorites list of the user.
Alternative flow:	NA
Postconditions:	The artist is stored in the Favorite list of the Registered User and can be viewed.

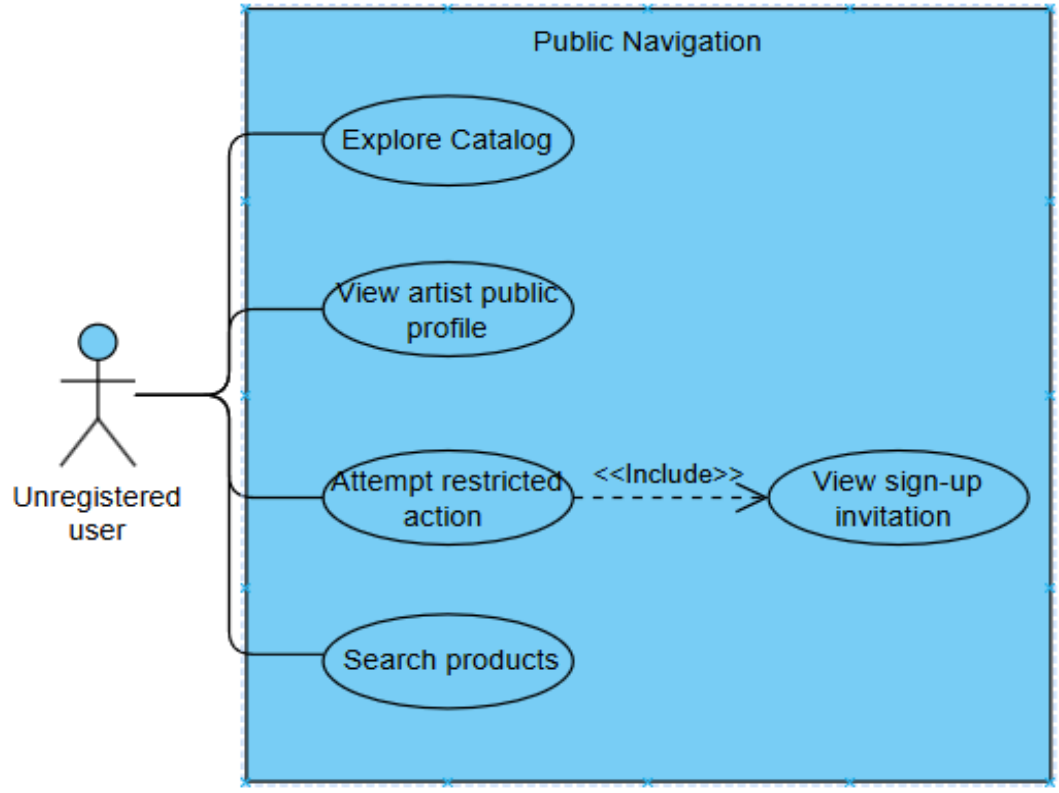
Use case name:	**CU-903: Remove artist from favorites**
Description:	Allows the user to remove an artist from their favorites list.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in. The artist is already in the favorite list. The user is located in the favorite list page.
Main flow:	1. The Registered User clicks on the “Remove from favorites” (Bookmark Icon). 2. The system deletes the relationship between the Registered User with the Artist ID from the database. 3. The system updates the user interface and removes the artist from the favorite list.
Alternative flow:	● A1: The user removes the artist from the favorite list while browsing. ○ The user is able to remove the artist while exploring different products or the artist section.
Postconditions:	The relationship between Registered User and Artist is deleted in the

	database. The artist will not appear in the user's favorite list.

Use case name:	**CU-904: Add product to favorites**
Description:	Allows the user to add a product to their favorite list.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in. The user is viewing a product.
Main flow:	1. The Registered User selects the option "Add product to favorites" (Heart Icon). 2. The system stores the relationship between the Registered User ID and the product ID in the database. 3. The system updates the favorite list of the Registered User.
Alternative flow:	NA
Postconditions:	The product is stored in the Favorite list of the Registered User and can be viewed.

Use case name:	**CU-905: Remove product from favorites**
Description:	Allows the user to remove a product from their favorite list.
Actors:	Registered user, Database
Preconditions:	The user must have an existing account and be logged in. The product is already in the favorites list. The user is located in their favorite list page.
Main flow:	1. The Registered User selects the "Remove product from favorites" (click on the heart icon). 2. The system deletes the relationship between the Register User ID and the product ID. 3. The system updates the user interface and removes the product from the favorite list.
Alternative flow:	● A1: The user removes the product while browsing. ○ The user is able to remove the product while browsing and selecting the favorite product and remove the relationship in the product page.
Postconditions:	The relationship between Registered User and Product is deleted in the database. The product will not appear in the user's favorite list.

2.2.8 Module 7: Public Navigation



Use case name:	**CU-1201: Browsing Without Registration**
Description:	As an unregistered user, I want to explore the platform.
Actors:	Unregistered user
Preconditions:	• The user opens the platform (Web or mobile) without being logged in. • The system is online and available.
Main flow:	1. Unregistered users open the Reborn main page. 2. The system displays the public interface with navigation options, search bar, and catalog access. 3. Browse through the product catalog. 4. Opens an artist profile from the catalog or search results. 5. Try to do a restricted action (add to cart). 6. The system shows a message inviting the user to sign up or log in.
Alternative flow:	• A1: No products available The system shows a message saying there are no products available. • A2: Connection error The system shows an error message saying content could not be loaded.

Postconditions:	● The user can see the public content of the platform. ● If the user tries to do restricted action, the system shows a message inviting them to sign up or log in.

2.3 User Characteristics

2.3.1 General Users

General users or guest users are only able to use basic functionalities in the system, such as:

- View products available
- View artists' profiles
- Search for products and use filters

2.3.2 Registered Users

Registered users are able to use the same functionalities that guest users can use, in addition to:

- Purchase or offer products
- Modify their profile
- View their completed or pending orders

2.4 Constraints

Technological constraints:

- The system must be deployed on AWS infrastructure (Amplify, EC2, S3, RDS, etc.).
- The system must have an SSL certificate.

External constraints:

- The system must use a payment gateway (Stripe).
- Product prices must be displayed exclusively in Mexican pesos (MXN).

Security constraints:

- User passwords must be securely stored using hashing and salting techniques.

- The system must temporarily lock accounts after multiple failed login attempts.

Availability constraints:

- The system must be available 99% of the time; the remaining time will be used for maintenance.
- The system must not be down for more than 30 minutes due to any incident.
- The system must be available at all times and operational during payment processes.

Performance constraints:

- The system must support at least 100 simultaneous users at the beginning of implementation.
- All platform pages must load in less than 3 seconds.

Usability / Design constraints:

- The system must be responsive for other devices.
- The platform design must be responsive, correctly adapting to desktop, tablet, and mobile devices.

2.5 Assumptions and dependencies

2.5.1 Assumptions

ID	Description	If assumption is false	Possible solution
A1	Users have internet access to use the platform.	Users cannot access the platform.	Provide offline caching for certain content.
A2	Artists provide accurate product and profile information.	Incorrect or incomplete information is displayed to users.	Implement content validation, moderation.
A3	Stripe will always be available for payment processing.	Payment transactions fail.	Integrate alternative payment gateway
A4	AWS services are operational.	Platform experiences downtime	Implement failover strategies

A5	Users' devices support modern web technologies	Platform UI breaks on some devices.	Test across devices, implement responsive design
A6	Users understand how to navigate the platform.	Users experience confusion or frustration.	Provide onboarding guides, tooltips, and intuitive UI design.

2.5.2 Dependencies

ID	Dependency	Purpose	Alternative
D1	AWS Amplify	Host and deploy the front-end application	Azure Static Web Apps
D2	AWS S3	Store static assets like images or videos.	Azure Blob Storage
D3	AWS API Gateway	Provide RESTful API endpoints for communication between frontend and backend services.	Azure API Management
D4	AWS EC2	Run backend services and application servers.	Azure Virtual Machines
D5	AWS RDS	Managed relational database to store users, products, orders, and transactions metadata.	Azure SQL Database
D6	Stripe	Payment gateway integration for secure processing of credit/debit card payments.	PayPal, MercadoPago
D7	AWS CloudWatch	Monitoring, logging, and alerting for system health, performance, and security.	Datadog
D8	AWS IAM	Manage secure access and permissions for AWS services and resources.	Azure AD

2.6 Apportioning of requirements

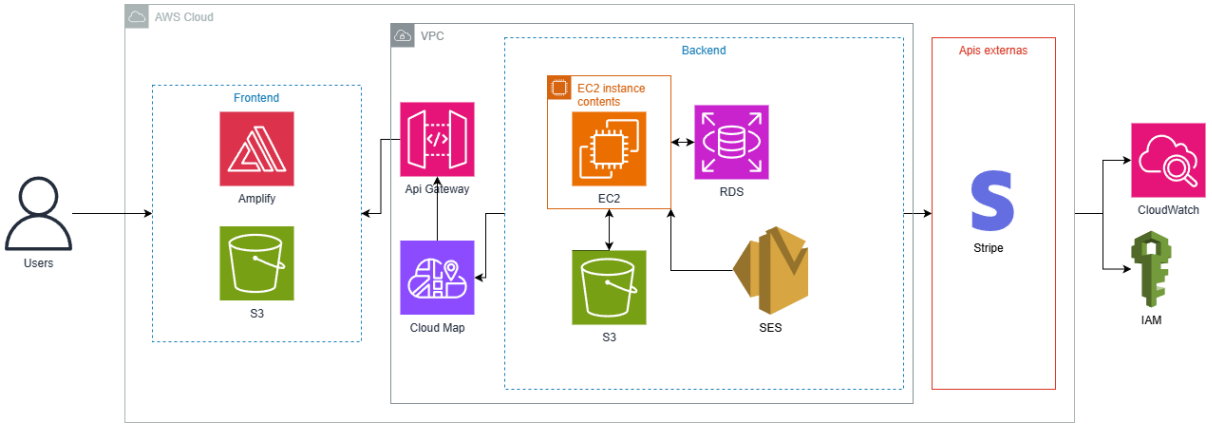
Use Case ID	Use Case Name	Primary Actor(s)	Priority
UC-101	Register	Unregistered user	**1**
UC-102	Log in	Registered user	**1**
UC-103	Password Recovery	Registered user	**1**
UC-104	Manage Account	Registered user	**2**
UC-105	Delete Account	Registered user	**3**
UC-106	Validate Registration	Unregistered user, Database	**1**
UC-107	Validate Fields	Registered user, Database	**1**
UC-201	Create product	User, database	**3**
UC-202	Validate image format	User, database	**1**
UC-203	Edit product	User, database	**2**
UC-204	Search products	User, guest, database	**2**
UC-205	Apply filters	User, guest, database	**1**
UC-206	View artist profile	User, guest, database	**1**
UC-504	Mark as favorite	Registered user	**3**
UC-701	View orders	Registered user	**1**
UC-702	Manage orders	Registered user	**1**
UC-703	Filter orders	Registered user	**2**
UC-707	View order details	Registered user	**1**
UC-801	Leave product review	User, database	**1**
UC-802	Validate purchase	Database, system	**2**
UC-803	View product review	User, guest, database	**2**
UC-804	Filter reviews	User, database	**1**
UC-901	View Favorite List	Registered user	**3**
UC-902	Add artist to favorites	Registered user	**3**

UC-903	Remove artist from favorites	Registered user	**3**
UC-904	Add product to favorites	Registered user	**3**
UC-905	Remove product from favorites	Registered user	**3**
UC-1201	Browsing Without Registration	Unregistered user	**2**

3 Specific requirements

3.1 External interfaces

3.1.1 AWS diagram



3.1.1.1 AWS Amplify

Description: AWS Amplify is a development platform for building source, scalable mobile and web applications. It provides a continuous deployment and hosting service for the frontend application.
Interface type: Frontend Hosting and Deployment.
Purpose:

- Host React frontend application
- Provide continuous deployment from version control systems
- Manage SSL certificates and custom domains
- Serve static assets and enable server-side rendering

Integration points:

- Connected to Amazon CloudFront for content delivery
- Integrates with Amazon S3 for asset storage
- Communicates with API Gateway for backend services

3.1.1.2 Amazon S3 (Simple Storage Service)

Description: Amazon S3 is an object storage service that offers industry-leading scalability, data availability, security, and performance.
Interface type: Object Storage
Purpose:

- Store and retrieve images and artist portfolios
- Host status website assets
- Maintain backup files and application logs
- Serve media content through CloudFront

Integration points:

- Frontend access S3 via CloudFront for optimized delivery

- Backend EC2 instances upload and manage objects
- Lambda functions process and optimize stored images

3.1.1.3 Amazon EC2 (Elastic Compute Cloud)

Description: Amazon EC2 provides scalable computing capacity in the AWS cloud, allowing users to run virtual servers.
Interface type: Backend Application Hosting
Purpose:

- Host the FastAPI backend application
- Process business logic and serve dynamic content
- Manage database connections and background tasks

Integration Points:

- Receives API request through API Gateway
- Connects to RDS PostgreSQL database
- Interacts with S3 for file storage
- Communicates with Lambda functions for specific tasks

3.1.1.4 Amazon RDS (Relational Database Service)

Description: Amazon RDS is a managed relational database service that provides easy setup, operation, and scaling of databases in the cloud.
Interface type: Database Management
Purpose:

- Store application data including users, products, orders, and transactions.
- Provide high availability through multi-AZ deployment
- Maintain data consistency and integrity

Integration points:

- EC2 instances connect for data persistence
- Lambda functions access for specific queries
- Database runs private subnet for security

3.1.1.5 Amazon CloudWatch

Description: Amazon CloudWatch is a monitoring and observability service that provides data and actionable insights for AWS resources and applications.
Interface type: Monitoring and logging
Purpose:

- Monitoring application performance and resource utilization
- Collect and track metrics from all AWS services
- Store and analyze application logs
- Set and automate responses to operational events

Integration points:

- Collects metrics from all AWS services
- Receives custom metrics from EC2 services
- Monitors Lambda function execution and performance
- Tracks API Gateway usage and errors

3.1.1.6 AWS IAM (Identity and Access Management)

Description: AWS IAM enables management of access to AWS services and resources securely. It controls authentication and authorization for AWS services.
Interface type: Security and Access Control
Purpose:

- Manage user permissions and access polices
- Control which services can interact with each other
- Audit API usage and access patterns

Integration points:

- Applied to all AES services for access and control
- Manages roles for EC2 instances and Lambda functions
- Controls S3 bucket policies and RDS security groups

3.1.2 External Payment Gateway

3.1.2.1 Stripe Payment Platform

Description: Stripe is a technology company that builds economic infrastructure for the internet. Businesses of every size use its software to accept payments and manage their business online.
Interface type: Payment Processing API
Purpose:

- Process credit and debit card payments securely
- Handle payment authentications and authorization
- Manage payment intents and confirmations
- Process refunds
- Ensure PCI DSS compliance

Integration points:

- Lambda functions interact with Stripe API for payment processing
- Frontend integrates Stripe.js for secure client-side tokenization
- Webhook endpoints receive payment status updates
- All payment data handled externally (no sensitive data stored in our systems)

Security considerations:

- All payment transactions use HTTPS encryption
- No sensitive payment information stored in our database
- Stripe handles PCI DSS compliance requirements
- Webhook signatures verified to ensure data integrity

3.1.3 Figma

Figma

3.2 Functions

RF-001
Description: “Users shall be able to view public artist profiles, including biography and product portfolio.”
Type: Functional Requirement.
Criterion: Ensure artist profiles and portfolios display all public information correctly.

RF-002
Description: “Users shall be able to upload content to their personal catalog, including descriptions and images.”
Type: Functional Requirement.
Criterion: Ensure users can upload valid images and descriptions without errors.

RF-003
Description: “Users shall be able to use the application without logging in, with access limited to viewing content only.”
Type: Functional Requirement.
Criterion: Ensure guests can browse content but cannot perform transactions.

RF-004
Description: “The system shall allow users to make purchases by specifying delivery method, payment method, and order details.”
Type: Functional Requirement.
Criterion: Ensure complete and secure purchase flow with all required data.

RF-005
Description: “The system shall allow account creation by requesting a unique username, password, and email.”
Type: Functional Requirement.
Criterion: Ensure new accounts are created with unique and verified credentials.

RF-006
Description: “The system shall allow users to permanently delete their account.”
Type: Functional Requirement.
Criterion: Ensure account deletion removes all related data as intended.

RF-007
Description: “The system shall require email and password for user login.”
Type: Functional Requirement.
Criterion: Ensure only valid credentials grant platform access.

RF-008
Description: “The system shall allow password recovery using the user’s email and username.”
Type: Functional Requirement.
Criterion: Ensure password resets are processed securely through email verification.

RF-009
Description: “The system may allow users to add additional profile information such as birth date, phone number, profile picture, and biography.”
Type: Functional Requirement.
Criterion: Ensure users can add or update optional profile data safely.

RF-010
Description: “The system shall enforce minimum password security requirements.”
Type: Functional Requirement.
Criterion: Ensure passwords include uppercase, lowercase, numbers, symbols, and at least 12 characters.

RF-012
Description: “The system may store user transaction records.”
Type: Functional Requirement.
Criterion: Ensure complete and accurate transaction logs are maintained.

RF-013
Description: “The system shall not store any user payment information.”
Type: Functional Requirement.
Criterion: Ensure compliance with payment security standards and no payment data retention.

RF-014
Description: “The system shall provide a product search feature with filters by name, category, or artist.”
Type: Functional Requirement.
Criterion: Ensure users can locate products efficiently through multiple filters.

RF-015
Description: “The system shall allow buyers to add products to a shopping cart.”
Type: Functional Requirement.
Criterion: Ensure items can be added or removed from the cart without errors.

RF-016
Description: “The checkout process shall be secure and integrated with a payment gateway (Stripe) that accepts credit and debit cards.”
Type: Functional Requirement.
Criterion: Ensure payment processing is secure and reliable through Stripe integration.

RF-017
Description: “The system shall provide sellers with a dashboard to create, edit and manage the availability of products.”
Type: Functional Requirement.
Criterion: Ensure sellers can fully manage their product listings.

RF-018
Description: “The system shall allow buyers to leave reviews on purchased products.”
Type: Functional Requirement.
Criterion: Ensure reviews can only be submitted for verified purchases.

RF-019
Description: “The system shall allow sellers to view their pending orders and their historical orders.”
Type: Functional Requirement.
Criterion: Ensure sellers can access accurate and updated order information.

RF-020
Description: “The system shall allow management of order statuses (pending, accepted or in process).”
Type: Functional Requirement.
Criterion: Ensure order statuses update accurately in real time.

RF-021
Description: “Users shall be able to add products or artists to a favorites list.”
Type: Functional Requirement.
Criterion: Ensure users can manage and store favorite items successfully.

RF-022
Description: “Only registered users shall be able to purchase products.”
Type: Functional Requirement.
Criterion: Ensure purchase functions are restricted to authenticated users.

RF-023
Description: “Registered users who have purchased a product shall be able to leave a review.”
Type: Functional Requirement.
Criterion: Ensure only verified buyers can submit reviews.

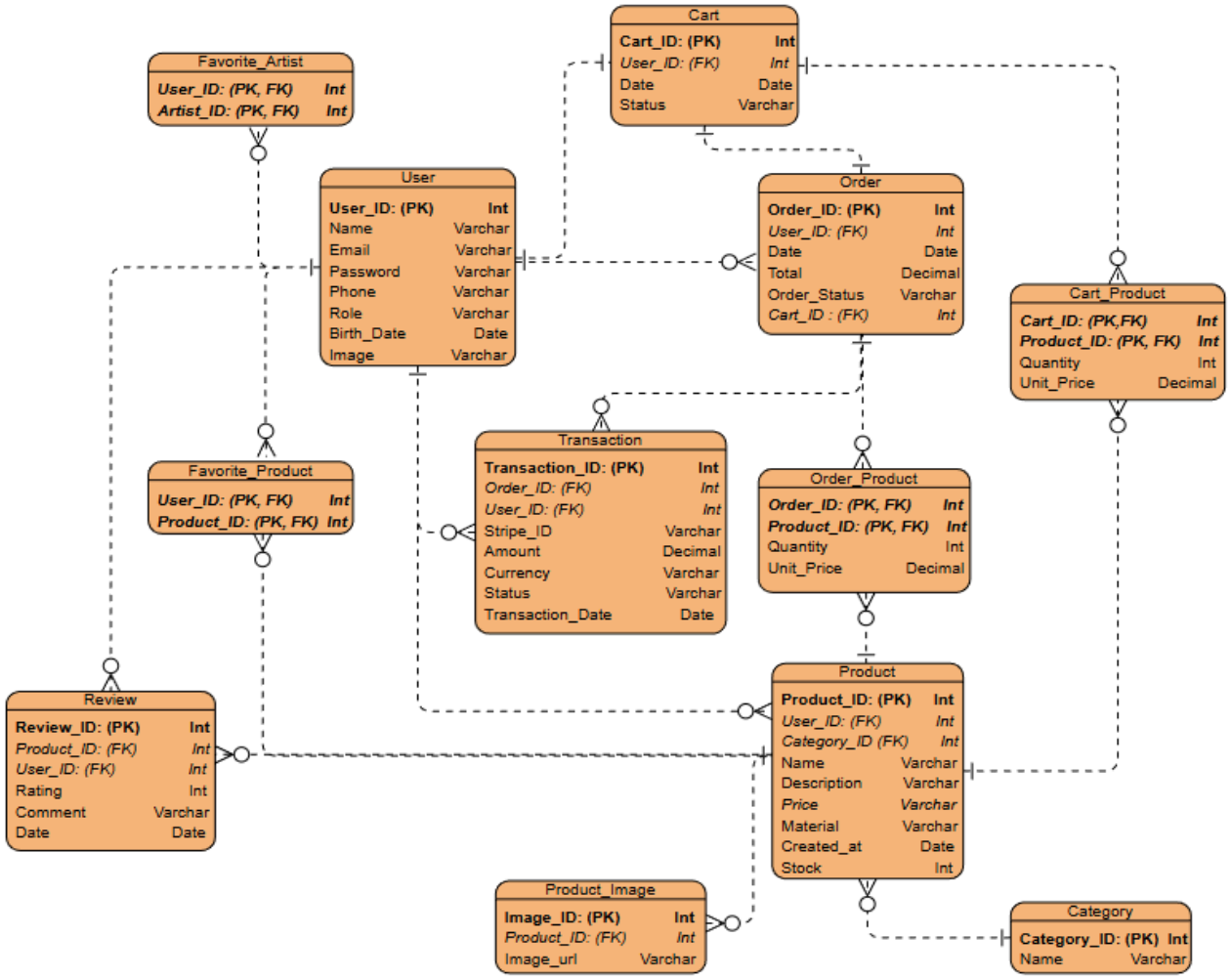
3.3 Performance requirements

RNF-06
Description: “The system must support at least 100 simultaneous users at the beginning of implementation.”
Type: Non-functional requirement (Performance)
Criterion: Ensure that the system can support a certain number of users at the same time.

RNF-011
Description: “All platform pages must load in less than 3 seconds.”
Type: Non-functional requirement (Performance)
Criterion: Ensure that the system does not take longer than a certain amount when loading a new page.

3.4 Logical database requirements

3.4.1 Entity relationship diagram



3.4.2 ER - Description

User:

Primary Key: User_ID

Relationship:

User—Order **1:N**

User—Cart **1:1**

User—Product **1:N**

User—Favorite_artist **1:N**

User—Favorite_product **1:N**

User—Review **1:N**

User—Transaction **1:N**

Cart:

Primary Key: Cart_ID

Foreign Key: User_ID

Relationship:

Cart—User **1:1**

Cart—Cart_Product **1:N**

Cart—Order **1:1**

Order:

Primary Key: Order_ID

Foreign Key: User_ID, Cart_ID

Relationship:

Order—User **N:1**

Order—Order_Product **1:N**

Order—Cart **1:1**

Order—Transaction **1:N**

Product:

Primary Key: Product_ID

Foreign Key: User_ID**,** Category_ID

Relationship:

Product—Order_Product **1:N**

Product—Cart_Product **1:N**

Product—Category **N:1**

Product—User **N:1**

Product—Favorite **1:N**

Product—Review **1:N**

Product—Product_image **1:N**

Product_image:

Primary Key: Image_ID

Foreign Key: Product_ID

Relationship:

Product_image—Product **N:1**

Transaction:

Primary Key: Transaction_ID

Foreign Key: Order_ID, User_ID

Relationship:

Transaction—User **N:1**

Transaction—Order **N:1**

Category:

Primary Key: Category_ID

Relationship:

Category—Product **1:N**

Order_Product:

Primary Key: Order_ID, Product_ID

Foreign Key: Order_ID, Product_ID

Relationship:

Order_Product—Order **N:1**

Order_Product—Product **N:1**

Cart_Product:

Primary Key: Cart_ID, Product_ID

Foreign Key: Cart_ID, Product_ID

Relationship:

Cart_Product—Cart **N:1**

Cart_Product—Product **N:1**

Review:

Primary Key: Review_ID

Foreign Key: Product_ID, User_ID

Relationship:

Review—User **N:1**

Review—Product **N:1**

Favorite_Product:

Primary Key: User_ID **,** Product_ID

Foreign Key: User_ID **,** Product_ID

Relationship:

Favorite—User **N:1**

Favorite—Product **N:1**

Favorite_Artist:

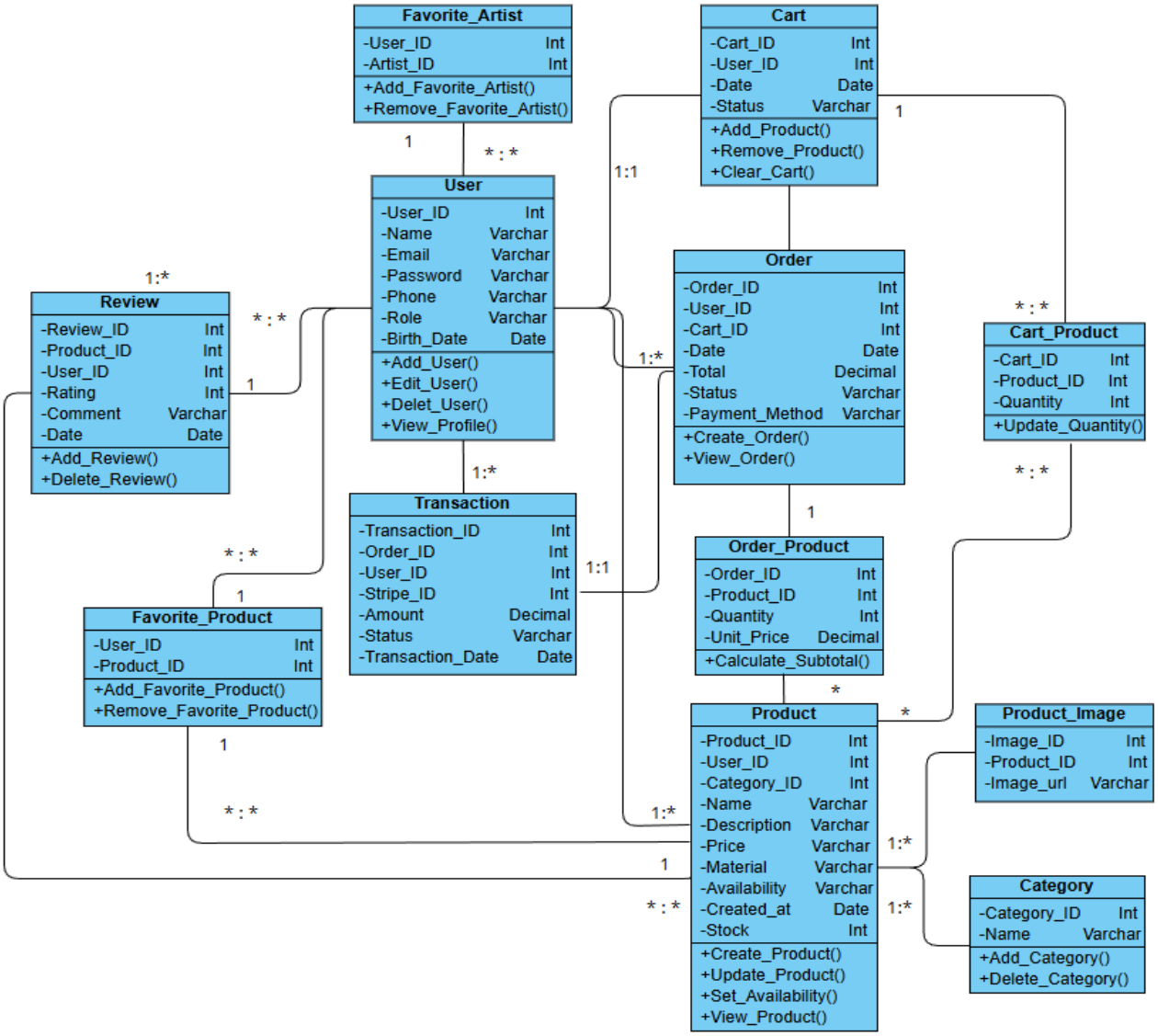
Primary Key: User_ID , Artist_ID (User_ID)

Foreign Key: User_ID , Artist_ID (User_ID)

Relationship:

Favorite—User **N:1**

3.4.2 Class Diagram



Descriptions

Class: User
Description: Represents the users of the platform, including buyers, sellers, and administrators.

Attributes:
- User_ID : Int — Unique identifier for the user.
- Name : Varchar — User's full name.
- Email : Varchar — User's email address.
- Password : Varchar — Encrypted password for login.
- Phone : Varchar — User's contact number.
- Role : Varchar — Defines the role (e.g., buyer, seller, admin).

- Birth_Date : Date — User’s date of birth.
Methods:
+ Add_User() — Adds a new user to the system.
+ Edit_User() — Updates user information.
+ Delete_User() — Removes a user account.
+ View_Profile() — Displays user profile details.

Class: Product
Description: Represents a product listed on the platform by a seller or artisan.

Attributes:
- Product_ID : Int — Unique product identifier.
- User_ID : Int — ID of the seller who created the product.
- Category_ID : Int — ID of the related category.
- Name : Varchar — Product name.
- Description : Varchar— Detailed description of the product.
- Price : Varchar— Price of the product.
- Material : Varchar— Material used for the product.
- Create_at : Date — Date the product was created.
- Stock : Int — Available stock quantity.

Methods:
+ Create_Product() — Registers a new product.
+ Update_Product() — Modifies product details.
+ Set_Availability(status: Varchar) — Changes visibility (e.g., 'Available', 'Out of Stock').
+ View_Product() — Displays product information.

Class: Product_Image
Description: Links a product to its image URLs. Represents the images associated with a product listing.

Attributes:
- Image_ID: Int — Unique image identifier.
- Product_ID: Int — Linked product ID.
- Image_url: Varchar — URL or path to the stored image.

Class: Category
Description: Represents product categories such as jewelry, ceramics, textiles, etc.

Attributes:
- Category_ID : Int — Unique category identifier.
- Name : Varchar — Name of the category.

Methods:
+ Add_Category() — Adds a new category.
+ Delete_Category() — Removes a category.

Class: Cart
Description: Represents a user's shopping cart that holds selected products before checkout.

Attributes:
- Cart_ID : Int — Unique cart identifier.
- User_ID : Int — ID of the user who owns the cart.
- Date : Date — Creation or last update date.
- Status : Varchar — Current state of the cart.

Methods:
+ Add_Product() — Adds a product to the cart.
+ Remove_Product() — Removes a product from the cart.
+ Clear_Cart() — Empties all products from the cart.

Class: Cart_Product
Description: Represents the relationship between carts and products, including quantities.

Attributes:
- Cart_ID : Int — Linked cart ID.
- Product_ID : Int — Linked product ID.
- Quantity : Int — Quantity of the product in the cart.

Methods:
+ Update_Quantity() — Updates product quantity in the cart.

Class: Order
Description: Represents a completed purchase made by a user.

Attributes:
- Order_ID : Int — Unique order identifier.
- User_ID : Int — ID of the user who placed the order.
- Date : Date — Order creation date.
- Total : Decimal — Total order cost.
- Status : Varchar — Current status (e.g., pending, shipped, completed).
- Payment_Method : Varchar — Payment method used.
- Cart_ID : Int (FK) — ID of the cart that generated the order.

Methods:
+ Create_Order() — Creates a new order.
+ View_Order() — Displays order details.

Class: Order_Product
Description: Represents products included in a specific order with their quantities and prices.

Attributes:
- Order_ID : Int — Linked order ID.
- Product_ID : Int — Linked product ID.
- Quantity : Int — Number of units ordered.
- Unit_Price : Decimal — Price per product unit.

Methods:
+ Calculate_Subtotal() — Calculates the subtotal for a product in the order.

Class: Transaction
Description: Records the financial details of the payment process.

Attributes:
- Transaction_ID : Int — Unique identifier for the transaction.
- Order_ID : Int — The order ID associated with this transaction.
- User_ID : Int — The user ID who initiated the transaction.
- Stripe_ID : Varchar — The unique ID generated by the Stripe payment gateway.
- Amount : Decimal — The monetary amount of the transaction.
- Status : Varchar — The current status of the payment (e.g., success, failed, pending).- Transaction_Date : Date — The date the transaction occurred.

Class: Review
Description: Represents a customer’s review and rating for a purchased product.

Attributes:
- Review_ID : Int — Unique review identifier.
- Product_ID : Int — Reviewed product ID.
- User_ID : Int — Reviewer’s user ID.
- Rating : Int — Rating given to the product.
- Comment : String — Written feedback.
- Date : Date — Date of the review.

Methods:
+ Add_Review() — Adds a new review.
+ Delete_Review() — Removes a review.

Class: Favorite_Product
Description: User's favorite products (N:M association).

Attributes:
- User_ID : Int — User who marked the favorite.
- Product_ID : Int — Product marked as favorite.

Methods:
+ Add_Favorite_Product()
+ Remove_Favorite_Product()

Class: Favorite_Artist
Description: User's favorite products (N:M association).

Attributes:
- User_ID : Int — User who marked the favorite.
- Artist_ID: Int — Artist marked as favorite.

Methods:
+ Add_Favorite_Artist()
+ Remove_Favorite_Artist()

User Relationships
User → Product
Type: One-to-Many
Description: A user (seller or artisan) can publish many products, but each product belongs to only one user.

User → Cart
Type: One-to-Many
Description: Each user can have one or more carts (for example, active or completed), but each cart belongs to a single user.

User → Order
Type: One-to-Many
Description: A user can place many orders, but each order is linked to only one user.

User → Review
Type: One-to-Many
Description: A user can write multiple reviews, but each review is written by a single user.

User → Transaction
Type: One-to-Many
Description: A user is associated with multiple transactions.

User → Favorite_Product
Type: Many-to-Many
Description: A user can mark many products as favorites; this association is handled by the Favorite_Product class.

User → Favorite_Artist
Type: Many-to-Many
Description: A user can mark many other users (artists) as favorites; this association is handled by the Favorite_Artist class.

Product Relationships
Product → Category
Type: Many-to-One
Description: Each product belongs to one category, but a category can contain many products.

Product → User
Type: Many-to-One
Description: Each product is created by one seller, but one seller can have many products.

Product → Review
Type: One-to-Many

Description: A product can have multiple reviews from different users, but each review belongs to one product.

Product → Favorite_Product
Type: Many-to-Many
Description: A product can be marked as favorite by many users.

Product → Order_Product
Type: Many-to-Many
Description: A product can appear in multiple orders, and each order can include multiple products.

Product → Cart_Product
Type: Many-to-Many (through Cart_Product)
Description: A product can be in many carts, and a cart can contain multiple products.

Product → Product_Image
Type: One-to-Many
Description: A product can be associated with multiple images.

Transaction Relationships

The Transaction class records the financial details of the payment process.

Transaction → User
Type: Many-to-One
Description: The transaction is linked to the user who initiated it.

Transaction → Order
Type: Many-to-One
Description: The transaction is linked to the specific order being paid for.

Cart Relationships
Cart → User
Type: Many-to-One
Description: Each cart belongs to a single user.

Cart → Cart_Product
Type: One-to-Many
Description: Each cart can contain multiple products, and this relationship is managed by Cart_Product.

Order Relationships

Order → User

Type: Many-to-One

Description: Each order is placed by one user, but a user can place many orders.

Order → Cart

Type: One-to-One

Description: An order is generated from one cart at the time of checkout.

Order → Order_Product

Type: One-to-Many

Description: Each order can include several products, managed by Order_Product.

Order → Transaction

Type: One-to-Many

Description: An order is linked to one or more transactions (e.g., payment attempts).

Review Relationships

Review → Product

Type: Many-to-One

Description: Each review refers to one product.

Review → User

Type: Many-to-One

Description: Each review is written by one user.

Category Relationships

Category → Product

Type: One-to-Many

Description: Each category can have multiple products, but each product belongs to one category.

Intermediate Relationship Classes

Order_Product: Connects Order and Product (many-to-many).

Cart_Product: Connects Cart and Product (many-to-many).

Favorite_Product: Connects User and Product (many-to-many).

Favorite_Artist: Connects User and User (Artist) (many-to-many).

Transaction: Connects Order and User to record payment details.

Product_Image: Links Product to its image URLs.

3.5 Design Constraints

The system is subject to the following design constraints:

3.5.1 Regulatory Constraints

- The system must use a payment gateway (Stripe) to ensure PCI DSS compliance for secure payment processing.
- The system must encrypt all sensitive user information, including passwords, using industry-standard hashing and salting techniques.
- The system must implement SSL certificates to secure all data transmissions.

3.5.2 System Constraints

- The system must maintain 99% uptime, with planned maintenance allowed.
- The system must be available at all times during payment processes.
- The system must support at least 100 simultaneous users at the start of implementation.
- The system must load all pages in less than 3 seconds.
- The system must be deployable on AWS infrastructure (Amplify, EC2, S3, RDS, etc.).
- The system must scale efficiently as user load and transactions increase.

3.5.4 User Interface Constraints

- The platform must be responsive and adapt correctly across desktop, tablet, and mobile devices.
- The user interface must provide consistent layout and functionality across all supported devices.

3.5.5 Localization Constraints

- The system must display all product prices exclusively in Mexican Pesos (MXN).

3.6 Software system attributes

3.6.1 Reliability

RNF-9: The system must not be down for more than 30 minutes.
RNF-5: The system must be available 99% of the time (planned maintenance allowed).

3.6.2 Availability

RNF-10: The system must be available at all times and operational during payment processes.
RNF-5: The system must be available 99% of the time (overlaps with reliability).

3.6.3 Security

RNF-3: The system must have an SSL certificate.
RNF-13: User passwords must be stored securely using hashing and salting.
RNF-14: The system must temporarily lock accounts after multiple failed login attempts.

3.6.4 Performance

RNF-6: The system must support at least 100 simultaneous users at the start of implementation.
RNF-11: All platform pages must load in less than 3 seconds.

3.6.5 Usability / Portability

RNF-7: The system must be responsive across devices.
RNF-12: The platform design must adapt correctly to desktop, tablet, and mobile.

3.6.6 Compliance / External Constraints

RNF-1: The system must be deployed on AWS infrastructure (Amplify, EC2, S3, RDS, etc.).
RNF-2: The system must use a payment gateway (Stripe).
RNF-8: Product prices must be displayed exclusively in MXN.

3.7 Organizing the specific requirements

3.7.1 Module 1: Authentication and User Management

Must
CU-101: As a new user, I want to register on the platform to access all functionalities.
CU-102: As a registered user, I want to log in to access my account.

Should
CU-103: As a user, I want to recover my password if I forget it.

Could
CU-104: As a user, I want to manage my personal information.

Won't
CU-105: As a user, I want to permanently delete my account.

3.7.2 Module 2: Catalog and Product Management

Must
CU-201: As an artist/seller, I want to create products in my catalog.
CU-202: As an artist, I want to edit my existing products.

Should
CU-203: As a buyer, I want to search products using different criteria.
CU-204: As a user, I want to view public artist profiles.

3.7.3 Module 3: Orders and Purchases Management

Must
CU-301: As a buyer, I want to place orders from the catalog.
CU-302: As a buyer, I want to complete my purchase securely.
CU-303: As a buyer, I want to manage products in my shopping cart.

3.7.4 Module 4: Order Status Management

Must
CU-701: As a seller, I want to manage the status of my orders.
CU-702: As a seller, I want to view my orders.

3.7.5 Module 5: Reviews and Ratings System

Must
CU-801: As a buyer, I want to leave reviews for purchased products.

Should
CU-802: As a user, I want to view product reviews.

3.7.6 Module 6: Public Navigation

Must
CU-1201: As an unregistered user, I want to explore the platform.

4 Supporting information

4.1 Table of contents and index

4.2 Appendixes

4.2.1 Gantt chart

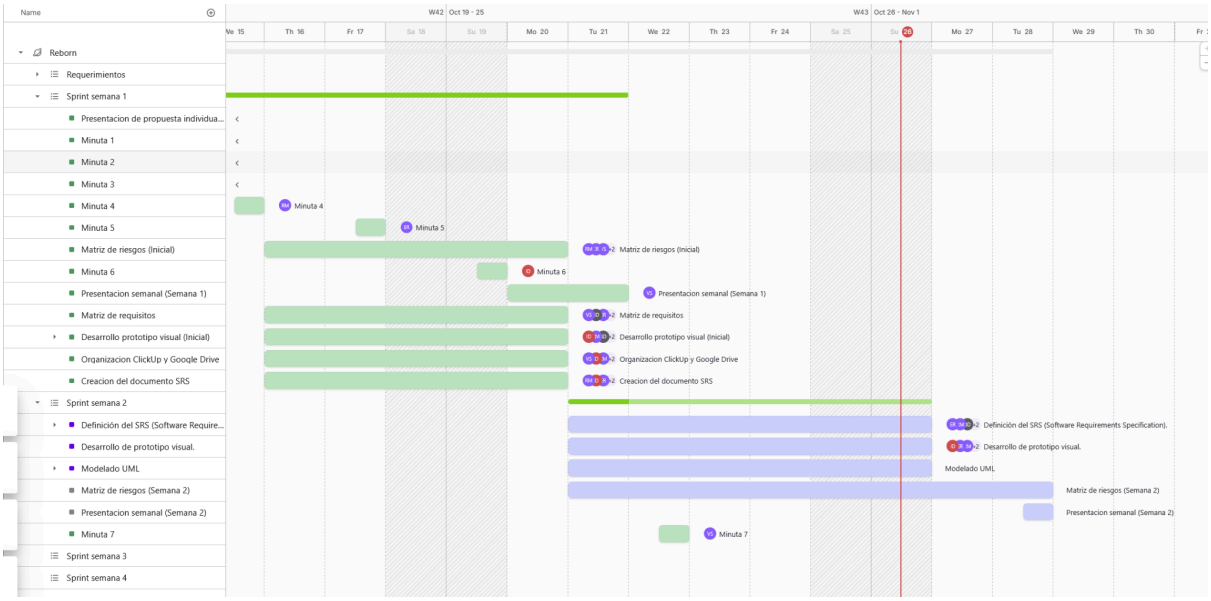
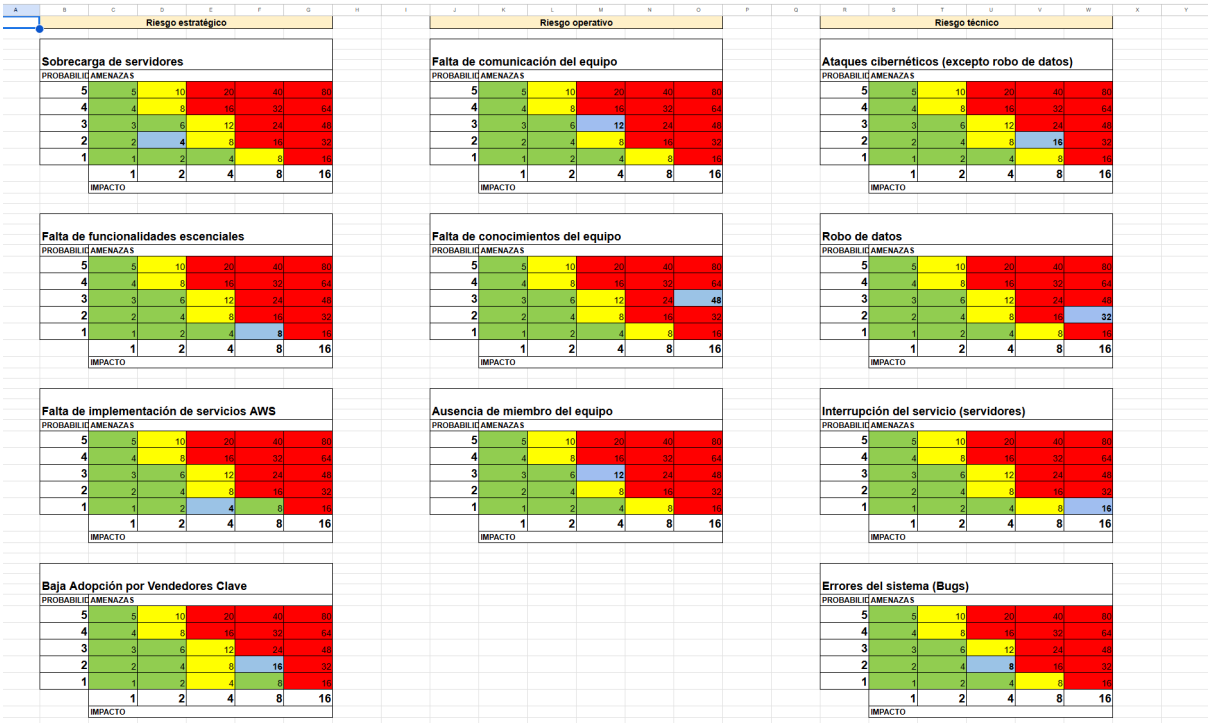


Diagrama de gantt / Matriz de riesgos

4.2.2 Matriz de riesgos



[Matriz de riesgos - Hojas de cálculo de Google]

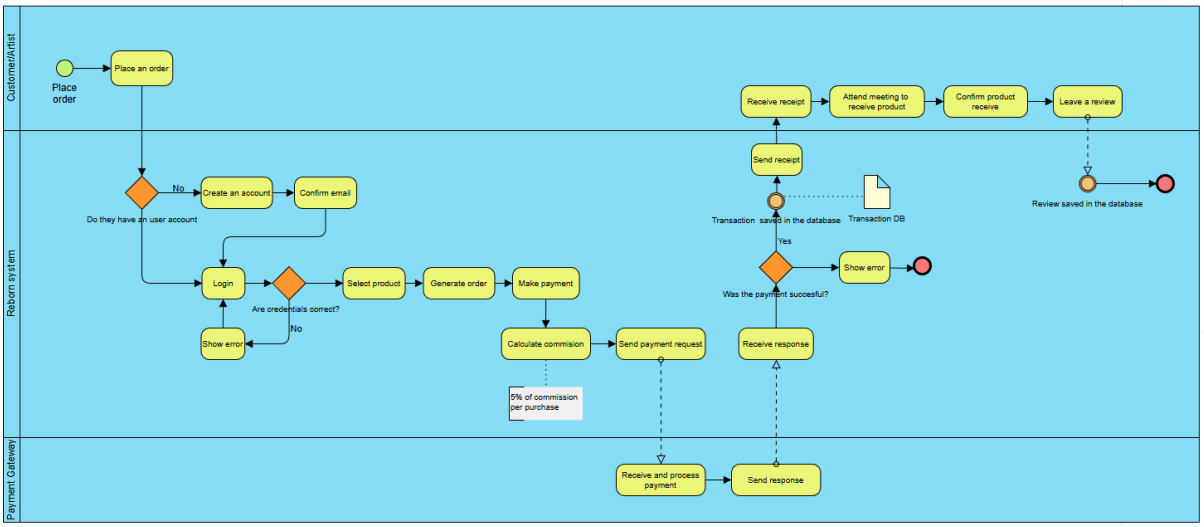
4.2.3 Requirements Traceability Matrix

Matriz

ID	Requirement Description	Type of Requirement	Key Functionality	Status	Priority Level	Parent Requirement	Potential Risks	Acceptance Criteria	Objectives and Goals
RF-1	Los usuarios deberán poder ver los perfiles públicos de los artistas, incluyendo su biografía y su portafolio de productos.	Functional Requirement	Profile viewing, portfolio display	Pending	Medium	-	Performance issues with image loading	Users can view artist profiles with all public information displayed correctly	Enhance artist visibility and user discovery
RF-2	El usuario debe poder subir contenido a su catálogo personal incluyendo: Descripciones e imágenes	Functional Requirement	Content upload, catalog management	Pending	High	RF-20	File upload failures, invalid formats	Users can successfully upload images and descriptions to their catalog	Enable artists to showcase their work
RF-3	El usuario debe poder realizar un pedido a un artista ya sea del catálogo del artista o un pedido personalizado.	Functional Requirement	Order placement, custom commissions	Pending	High	RF-5	Order processing errors, miscommunication	Users can place both catalog and custom orders successfully	Facilitate transactions between users and artists
RF-4	El usuario debe poder utilizar la aplicación sin iniciar sesión donde solo podrá visualizar el contenido pero no podrá comprar ni vender productos.	Functional Requirement	Guest browsing, limited access	Pending	Medium	-	Limited user engagement	Guest users can browse content but cannot access transactional features	Improve user acquisition and platform discovery
RF-5	El sistema debe permitir realizar compras. Debe solicitar: Forma de entrega, método de pago y orden.	Functional Requirement	Purchase processing, checkout	Pending	High	RF-3	Payment failures, order processing	Complete purchase flow with all required information	Enable secure and complete transactions
RF-6	El sistema debe permitir crear una cuenta. Debe solicitar: Nombre de usuario (username único), contraseña y correo	Functional Requirement	User registration, account creation	Pending	High	-	Duplicate usernames, email verification issues	Users can create accounts with unique credentials	Grow user base and enable personalized features
RF-7	El sistema debe permitir eliminar una cuenta	Functional Requirement	Account deletion, data management	Pending	Medium	RF-6	Accidental deletion, data retention issues	Users can permanently delete their accounts	Comply with data privacy regulations
RF-8	El sistema debe solicitar correo y contraseña para iniciar sesión.	Functional Requirement	User authentication, login	Pending	High	RF-6	Security breaches, unauthorized access	Secure login process with valid credentials	Ensure platform security and user privacy
RF-9	El sistema debe permitir restaurar la contraseña en caso de que esta se haya olvidado, utilizando el correo y nombre del usuario como método de recuperación.	Functional Requirement	Password recovery, email verification	Pending	Medium	RF-8	Security vulnerabilities in recovery process	Users can reset passwords via email verification	Reduce user friction and support issues
RF-10	El sistema debería enviar notificaciones al usuario.	Functional Requirement	Notification system, user alerts	Pending	Medium	-	Notification spam, delivery failures	Users receive timely and relevant notifications	Improve user engagement and communication
RF-11	El sistema podría permitir añadir información adicional al usuario. Fecha de nacimiento o edad, teléfono, foto de perfil y biografía.	Functional Requirement	Profile customization, additional data	Pending	Low	RF-6	Privacy concerns, data validation issues	Users can optionally add and update profile information	Enhance user profiles and personalization
RF-12	El sistema debería contar con requisitos mínimos de seguridad para la contraseña.	Functional Requirement	Password security, validation rules	Pending	High	RF-6	Weak passwords, security breaches	Una combinación de letras mayúsculas, minúsculas, números y símbolos. Tener al menos 12 caracteres	Enhance platform security
RF-13	El sistema debe permitir cancelar pedidos.	Functional Requirement	Order cancellation, order management	Pending	Medium	RF-3	Cancellation timing issues, refund processing	Users can cancel orders within allowed timeframes	Provide flexibility in order management
RF-14	El sistema podría almacenar las transacciones entre usuarios.	Functional Requirement	Transaction history, record keeping	Pending	Low	RF-5	Data storage issues, privacy concerns	Complete transaction records are maintained	Enable order tracking and dispute resolution
RF-15	El sistema podría permitir enviar y recibir mensajes de texto entre usuarios dentro de la misma plataforma.		Messaging system,				Spam, inappropriate content, privacy	Users can communicate	Facilitate artist-client

Matriz de requisitos

4.2.4 Business Process Modeling



This business process model diagram shows Reborn's business logic. If a customer wants to place an order, they must first log in or create an account in the system. Then, the customer can browse for products they wish to purchase.

After selecting and confirming the desired products, the system generates an order. At this point, the customer proceeds to make the payment. The system sends a payment request to the payment gateway. The payment gateway processes the payment and sends a response back to the system.

If the payment is successful, the system saves the transaction in the database and sends the receipt to both the customer and the artist. The artist receives the paid order and starts preparing the product for delivery. Then, they arrange the local delivery or pickup of the product.

After receiving the product, the customer confirms the delivery in the system and leaves a review, which is stored in the database